

Material extraído do livro

https://blog.aku.edu.tr/ismailkoyuncu/files/2017/04/02_ebook.pdf

3.7.4 Simplified floating-point adder

Floating point is another format to represent a number. With the same number of bits, the range in floating-point format is much larger than that in signed integer format. Although VHDL has a built-in floating-point data type, it is too complex to be synthesized automatically.

	sort	align	add/sub	normalize
eg. 1	+0.54E3	-0.87E4	-0.87E4	-0.87E4
	-0.87E4	+0.54E3	+0.05E4	+0.05E4
			-0.82E4	-0.82E4
eg. 2	+0.54E3	-0.55E3	-0.55E3	-0.55E3
	-0.55E3	+0.54E3	+0.54E3	+0.54E3
			-0.01E3	-0.10E2
eg. 3	+0.54E0	-0.55E0	-0.55E0	-0.55E0
	-0.55E0	+0.54E0	+0.54E0	+0.54E0
			-0.01E0	-0.00E0
eg. 4	+0.56E3	+0.56E3	+0.56E3	+0.56E3
	+0.52E3	+0.52E3	+0.52E3	+0.52E3
			+1.07E3	+0.10E4

Table 1: Floating-point addition examples.

Detailed discussion of floating-point representation is beyond the scope of this book. We use a simplified 13-bit format in this example and ignore the round-off error. The representation consists of a sign bit, s , which indicates the sign of the number (1 for negative); a 4-bit exponent field, e , which represents the exponent; and an 8-bit significand field, f , which represents the significand or the fraction. In this format, the value of a floating-point number is $(-1)^s \times 0.f \times 2^e$. The $0.f \times 2^e$ is the magnitude of the number and $(-1)^s$ is just a formal way to state that “ s equal to 1 implies a negative number.” Since the sign bit is separated from the rest of the number, floating-point representation can be considered as a variation of the sign-magnitude format.

We also make the following assumptions:

- Both exponent and significand fields are in unsigned format.
- The representation has to be either normalized or zero. Normalized representation means that the MSB of the significand field must be ‘1’. If the magnitude of the computation result is smaller than the smallest normalized nonzero magnitude, $0.10000000 \times 2^{0000}$, it must be converted to zero.

Under these assumptions, the largest and smallest nonzero magnitudes are $0.11111111 \times 2^{1111}$ and $0.10000000 \times 2^{0000}$, and the range is about 2^{16} (i.e., $0.10000000 \times 2^{0000} - 0.11111111 \times 2^{1111}$).

Our floating-point adder design follows the process of adding numbers manually in scientific notation. This process can best be explained by examples. We assume that the widths of the exponent and significand are 2 and digits, respectively. Decimal format is used for clarity. The computations of several representative examples are shown in Figure 3.8.

The computation is done in four major steps:

1. Sorting: puts the number with the larger magnitude on the top and the number with the smaller magnitude on the bottom (we call the sorted numbers “big number” and “small number”).
2. Alignment: aligns the two numbers so they have the same exponent. This can be done by adjusting the exponent of the small number to match the exponent of the big number. The significand of the small number has to shift to the right according to the difference in exponents.
3. Addition/subtraction: adds or subtracts the significands of two aligned numbers.

4. Normalization: adjusts the result to normalized format.

Three types of normalization procedures may be needed:

- After a subtraction, the result may contain leading zeros in front, as in example 2.
- After a subtraction, the result may be too small to be normalized and thus needs to be converted to zero, as in example 3.
- After an addition, the result may generate a carry-out bit, as in example 4.

Our binary floating-point adder design uses a similar algorithm. To simplify the implementation, we ignore the rounding. During alignment and normalization, the lower bits of the significand will be discarded when shifted out. The design is divided into four stages, each corresponding to a step in the foregoing algorithm. The suffixes, 'b', 's', 'a', 'r', and 'n', used in signal names are for “big number,” “small number,” “aligned number,” “result of addition/subtraction,” and “normalized number,” respectively. The code is developed according to these stages, as shown in Listing 3.19.

Listing 3.19 Simplified floating-point adder

```
1 library ieee; --
2 use ieee.std_logic_1164.all; --
3 use ieee.numeric_std.all; --
4 entity fp_adder is --
5     port ( --
6         sign1, sign2: in std_logic; --
7         exp1, exp2: in std_logic_vector (3 downto 0); --
8         frac1, frac2: in std_logic_vector(7 downto 0); --
9         sign_out: out std_logic; --
10        exp_out: out std_logic_vector (3 downto 0); --
11        frac_out: out std_logic_vector(7 downto 0) --
12    ); --
13 end fp_adder; --
14
15 architecture arch of fp_adder is --
16     -- suffix b, s, a, n for --
17     -- big, small, aligned, normalized number --
18     signal signb, signs: std_logic; --
19     signal expb, exps, expn: unsigned (3 downto 0); --
20     signal fracb, fracs, fracn: unsigned (7 downto 0); --
21     signal sum_norm: unsigned (7 downto 0); --
22     signal exp_diff: unsigned (3 downto 0); --
23     signal sum: unsigned (8 downto 0); -- one extra for carry --
24     signal leado: unsigned (2 downto 0); --
25 begin --
26
27     -- 1st stage: sort to find the larger number --
28     process (sign1, sign2, exp1, exp2, frac1, frac2) --
29     begin
30         if (exp1 & frac1) > (exp2 & frac2) then --
31             signb <= sign1; --
32             signs <= sign2; --
33             expb <= unsigned(exp1); --
34             exps <= unsigned(exp2); --
35             fracb <= unsigned(frac1); --
36             fracs <= unsigned(frac2); --
37         else --
38             signb <= sign2; --
39             signs <= sign1; --
40             expb <= unsigned(exp2); --
41             exps <= unsigned(exp1); --
42             fracb <= unsigned(frac2); --
43             fracs <= unsigned(frac1); --
44         end if; --
45     end process; --
```

```

46
47 -- 2nd stage: align smaller number --
48 exp_diff <= expb - exps; --
49 with exp_diff select --
50     fracb <= --
51         fracs when "0000", --
52         "0" & fracs(7 downto 1) when "0001", --
53         "00" & fracs(7 downto 2) when "0010", --
54         "000" & fracs(7 downto 3) when "0011", --
55         "0000" & fracs(7 downto 4) when "0100", --
56         "00000" & fracs(7 downto 5) when "0101", --
57         "000000" & fracs(7 downto 6) when "0110", --
58         "0000000" & fracs(7) when "0111", --
59         "00000000" when others; --
60
61 -- 3rd stage: add/subtract --
62 sum <= --
63     ('0' & fracb) + ('0' & fracb) when signb=signs else --
64     ('0' & fracb) - ('0' & fracb); --
65
66 -- 4th stage: normalize --
67 -- count leading 0s --
68 leado <= "000" when (sum(7)='1') else --
69     "001" when (sum(6)='1') else --
70     "010" when (sum(5)='1') else --
71     "011" when (sum(4)='1') else --
72     "100" when (sum(3)='1') else --
73     "101" when (sum(2)='1') else --
74     "110" when (sum(1)='1') else --
75     "111"; --
76
77 -- shift significand according to leading 0 --
78 with leado select --
79     sum_norm <= --
80         sum(7 downto 0) when "000", --
81         sum(6 downto 0) & '0' when "001", --
82         sum(5 downto 0) & "00" when "010", --
83         sum(4 downto 0) & "000" when "011", --
84         sum(3 downto 0) & "0000" when "100", --
85         sum(2 downto 0) & "00000" when "101", --
86         sum(1 downto 0) & "000000" when "110", --
87         sum(0) & "0000000" when others; --
88
89 -- normalize with special conditions --
90 process (sum, sum_norm, expb, leado) --
91 begin
92     if sum(8)='1' then --
93         expn <= expb + 1; --
94         -- w/ carry out; shift frac to right --
95         fracn <= sum(8 downto 1); --
96     elsif (leado > expb) then --
97         expn <= (others => '0'); --
98         fracn <= (others => '0'); --
99         -- too small to normalize. set to 0 --
100     else --
101         expn <= expb - leado; --
102         fracn <= sum_norm; --
103     end if; --
104 end process; --
105
106 -- form output --
107 sign_out <= signb; --
108 exp_out <= std_logic_vector(expn); --

```

```

109     frac_out <= std_logic_vector(fracn); --
110 end arch; --

```

The circuit in the first stage compares the magnitudes and routes the big number to the signb, expb, and fracb signals and the smaller number to the signs, exps, and fracs signals. The comparison is done between $\text{exp1} \& \text{frac1}$ and $\text{exp2} \& \text{frac2}$. It implies that the exponents are compared first, and if they are the same, the significands are compared. The circuit in the second stage performs alignment. It first calculates the difference between the two exponents, which is $\text{expb} - \text{exps}$, and then shifts the significand, fracs, to the right by this amount. The aligned significand is labeled frac_a. The circuit in the third stage performs sign-magnitude addition, similar to that in Section 3.7.2. Note that the operands are extended by 1 bit to accommodate the carry-out bit.

The circuit in the fourth stage performs normalization, which adjusts the result to make the final output conform to the normalized format. The normalization circuit is constructed in three segments. The first segment counts the number of leading zeros. It is somewhat like a priority encoder. The second segment shifts the significands to the left by the amount specified by the leading-zero counting circuit. The last segment checks the carry-out and zero conditions and generates the final normalized number.

Testing circuit The floating-point adder has two 13-bit input operands. Since the prototyping board has only one 8-bit switch and four 1-bit pushbuttons, it cannot provide enough number of physical inputs to test the circuit. To accommodate the 26 bits of the floating-point adder, we must create a testing circuit and assign constants or duplicated switch signals to the adder's input operands. An example is shown in Listing 3.20. It assigns one operand as constant and uses duplicated switch signals for the other operand. The addition result is passed to the hexadecimal decoders and the sign circuit and is shown on the seven-segment LED display.

Listing 3.20 Floating-point adder testing circuit

```

1  library ieee; --
2  use ieee.std_logic_1164.all; --
3  use ieee.numeric_std.all; --
4
5  entity fp_adder_test is --
6      port( --
7          clk: in std_logic; --
8          sw: in std_logic_vector(7 downto 0); --
9          btn: in std_logic_vector(3 downto 0); --
10         an: out std_logic_vector(3 downto 0); --
11         sseg: out std_logic_vector(7 downto 0) --
12     ); --
13 end fp_adder_test; --
14
15 architecture arch of fp_adder_test is --
16     signal sign1, sign2: std_logic; --
17     signal exp1, exp2: std_logic_vector(3 downto 0); --
18     signal frac1, frac2: std_logic_vector(7 downto 0); --
19     signal sign_out: std_logic; --
20     signal exp_out: std_logic_vector(3 downto 0); --
21     signal frac_out: std_logic_vector(7 downto 0); --
22     signal led3, led2, led1, led0: std_logic_vector(7 downto 0); --
23 begin --
24     -- set up the fp adder input signals --
25     sign1 <= '0'; --
26     exp1 <= "1000"; --
27     frac1 <= '1' & sw(1) & sw(0) & "10101"; --
28     sign2 <= sw(7); --
29     exp2 <= btn; --
30     frac2 <= '1' & sw(6 downto 0); --
31
32     -- instantiate fp adder --
33     fp_add_unit: entity work.fp_adder --
34         port map( --
35             sign1=>sign1, sign2=>sign2, exp1=>exp1, exp2=>exp2, --
36             frac1=>frac1, frac2=>frac2, --
37             sign_out=>sign_out, exp_out=>exp_out, --

```

```

38         frac_out=>frac_out --
39     ); --
40
41     -- instantiate three instances of hex decoders --
42     -- exponent --
43     sseg_unit_0: entity work.hex_to_sseg --
44         port map(hex=>exp_out, dp=>'0', sseg=>led0); --
45
46     -- 4 LSBs of fraction --
47     sseg_unit_1: entity work.hex_to_sseg --
48         port map(hex=>frac_out(3 downto 0), dp=>'1', sseg=>led1); --
49
50     -- 4 MSBs of fraction --
51     sseg_unit_2: entity work.hex_to_sseg --
52         port map(hex=>frac_out(7 downto 4), dp=>'0', sseg=>led2); --
53
54     -- sign --
55     led3 <= "11111110" when sign_out='1' else -- middle bar --
56         "11111111"; -- blank --
57
58     -- instantiate 7-seg LED display time-multiplexing module --
59     disp_unit: entity work.disp_mux --
60         port map( --
61             clk=>clk, reset=>'0', --
62             in0=>led0, in1=>led1, in2=>led2, in3=>led3, --
63             an=>an, sseg=>sseg --
64         ); --
65 end arch; --

```